FarmKonnect – AI-Powered Agricultural Marketplace

Final Year Project Report

University of Management and Technology

Department of Computer Science

FarmKonnect: AI-Powered Agricultural Marketplace for Pakistan

---

Submitted By:

Name	Roll Number
Ahtsham Adil	-
Javeria Zahid	-
Sameer Malik	-
Hamdan Aftab	-

Session: 2022-2026

Semester: 8th

Project Supervisor: Sir Abdul Jamil

Submission Date: January 2026

Table of Contents

---

1. Executive Summary

FarmKonnect is an AI-powered agricultural marketplace designed specifically for Pakistan's farming community. The platform addresses critical challenges faced by small and medium farmers, including lack of access to transparent market information, limited buyer reach, and price exploitation by middlemen (آڑھتی).

The system integrates three functional layers: 1. **Information Layer** - Real-time mandi prices from 15+ cities across Punjab 2. **Transaction Layer** - Direct farmer-to-buyer marketplace with real-time chat 3. **Intelligence Layer** - AI-based price forecasting for informed decision-making

Key Achievements

- ✅ Fully functional marketplace with CRUD operations for product listings
- ✅ Real-time chat system with offer/negotiation capabilities
- ✅ Automated price scraping from AMIS Pakistan (hourly updates)
- ✅ Interactive price charts with historical data visualization
- ✅ Secure cookie-based JWT authentication
- ✅ Admin panel for platform management
- ✅ Responsive design with dark/light theme support

Pending Features

- ⏳ AI price forecasting (ML model implementation)
- ⏳ Easypaisa/JazzCash payment integration
- ⏳ Push notifications and alerts
- ⏳ Bilingual support (Urdu/English)

---

2. Introduction

2.1 Background

Agriculture contributes approximately **19.2% to Pakistan's GDP** and employs **42.3% of the labor force**. Despite its significance, the agricultural sector faces systemic inefficiencies:

- **Information Asymmetry**: Farmers often lack access to real-time market prices, leading to price exploitation
- **Middlemen Exploitation**: Traditional آڑھتی (commission agents) take 8-15% commissions
- **Fragmented Markets**: No centralized platform connects farmers directly with buyers
- **Price Volatility**: Farmers cannot predict price movements to time their sales

2.2 Project Overview

FarmKonnect aims to transform agricultural trade in Pakistan by providing:

1. **Real-time Price Transparency**: Live mandi prices from AMIS Pakistan
2. **Direct Trade Platform**: Farmer-to-buyer marketplace eliminating middlemen
3. **AI-Powered Insights**: Price forecasting to help farmers make informed decisions
4. **Secure Transactions**: Digital payments via Easypaisa/JazzCash

2.3 Scope

Geographic Scope: - Currently limited to **Punjab, Pakistan** - 15 cities covered: Lahore, Faisalabad, Rawalpindi, Gujranwala, Multan, Sargodha, Sialkot, Bahawalpur, Rahim Yar Khan, Jhang, Layyah, Okara, Chichawatni, Bahawalnagar

Commodity Scope: - Wheat (گندم) - Rice (چاول) - Cotton (کپاس) - Sugar (چینی) - Maize (مکئی) - Flour (آٹا)

Future Expansion: - Other provinces (Sindh, KPK, Balochistan) - Additional commodities (vegetables, fruits, pulses)

---

3. Problem Statement

3.1 Core Problems

Pakistan’s agricultural sector faces the following challenges:

Problem	Impact	Statistics
Middlemen Exploitation	Farmers receive 40-60% of retail price	~Rs200B annual farmer losses
Price Information Gap	Farmers unaware of fair market prices	70% farmers rely on word-of-mouth
Fragmented Markets	No centralized trading platform	95% offline transactions
Price Volatility	Cannot predict optimal selling time	20-30% price variation weekly
Payment Delays	Cash-based, delayed settlements	30-60 days average payment cycle

3.2 Stakeholder Pain Points

Farmers: - Unable to access real-time mandi prices - Forced to accept middlemen’s quoted prices - No platform to reach buyers directly - Cannot predict price trends

Buyers (Mills, Traders, Retailers): - Difficulty finding verified suppliers - Quality verification challenges - Transportation coordination issues - No price benchmarking

3.3 Solution Approach

FarmKonnect addresses these problems through:

```
┌─────────────────────────────────────────────────────────┐
│                  FARMKONNECT SOLUTION                   │
├─────────────────────────────────────────────────────────┤
│                                                         │
│  Problem: Price Exploitation                            │
│  Solution: Real-time AMIS price display + Historical charts
│                                                         │
│  Problem: Middlemen Dependency                          │
│  Solution: Direct marketplace + In-app chat + Offers    │
│                                                         │
│  Problem: Price Uncertainty                             │
│  Solution: AI-powered 7-day price forecasting           │
│                                                         │
│  Problem: Payment Delays                                │
│  Solution: Easypaisa/JazzCash instant payments          │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

---

4. Objectives

4.1 Primary Objectives

1. **Enable Price Transparency**
   - Display real-time mandi prices from 15+ cities
   - Provide historical price charts and trends
   - Show price comparisons across cities
2. **Facilitate Direct Trade**
   - Allow farmers to list produce with photos
   - Enable buyers to browse and filter listings
   - Provide real-time chat for negotiations
3. **Implement AI Forecasting**
   - Predict commodity prices for next 7 days
   - Provide confidence intervals for predictions
   - Integrate external factors (weather, seasons)
4. **Ensure Secure Transactions**
   - Integrate Easypaisa/JazzCash payments
   - Generate digital receipts
   - Maintain transaction audit trail

4.2 Secondary Objectives

- Provide bilingual interface (Urdu/English)
- Send price alerts and notifications
- Enable admin monitoring and analytics
- Support mobile-responsive design

4.3 Success Metrics

Metric	Target	Current Status
User Registration	1000+ farmers	In Development
Daily Active Users	500+	In Development
Price Data Freshness	< 2 hours old	✅ Hourly scraping
Forecast Accuracy (MAPE)	< 10%	Not Implemented
Transaction Volume	Rs10M+ monthly	Not Implemented

---

5. Literature Review

5.1 Existing Solutions in Pakistan

Platform	Features	Limitations
AMIS Pakistan	Raw price data	No trading, no predictions
Pakissan.com	Agricultural news	No marketplace, outdated
GrowInsta	Farm management	No price data, limited adoption
Tazah Technologies	B2B produce trade	High-end focus, not farmer-centric

5.2 International Benchmarks

Platform	Country	Key Features
eNAM	India	National marketplace, price discovery
Alibaba Rural Taobao	China	Direct-to-consumer, logistics
Farmcrowdy	Nigeria	Crowdfunding + marketplace
Twiga Foods	Kenya	B2B produce, mobile-first

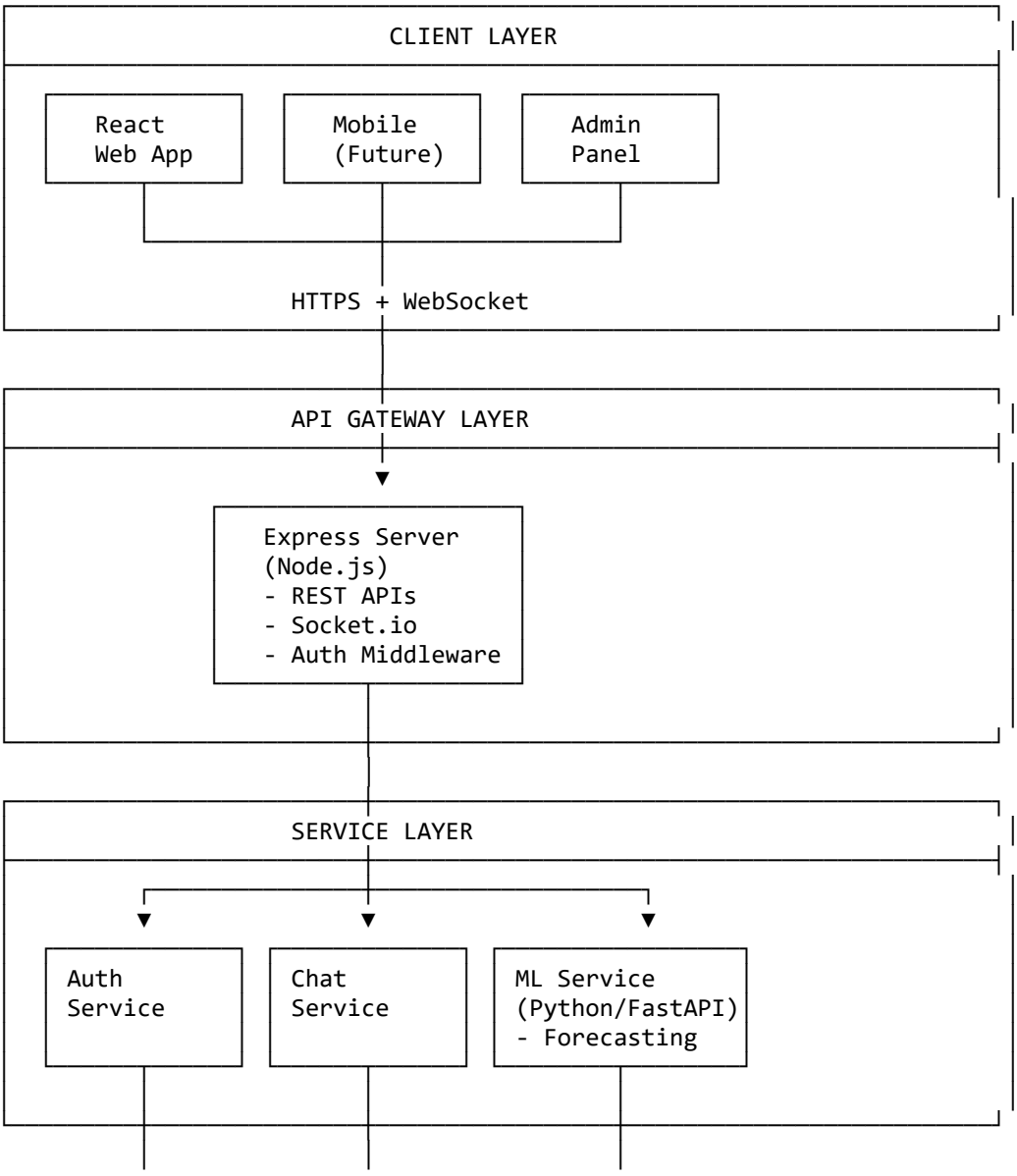
5.3 Technology Trends

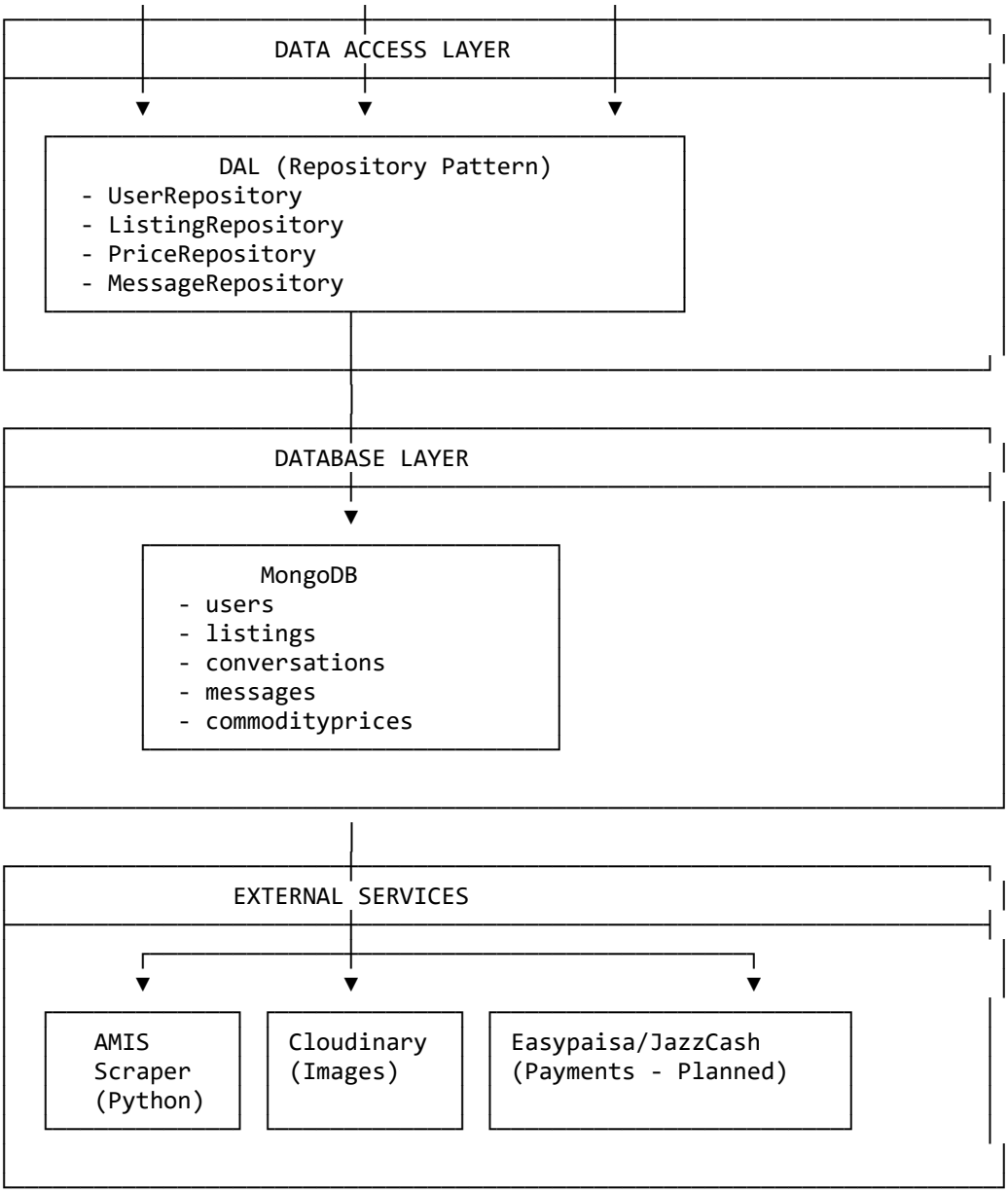
- **AI/ML in Agriculture**: Price forecasting using time series models
- **Mobile Payments**: Mobile money adoption in developing countries
- **Real-time Data**: WebSocket-based live updates
- **Microservices**: Modular, scalable architecture

---

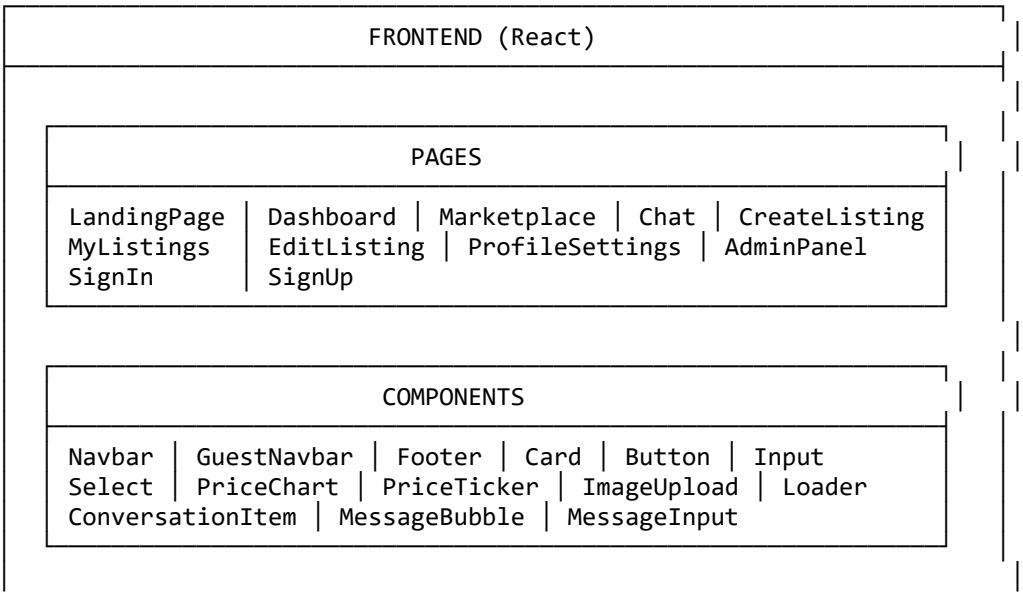
6. System Architecture

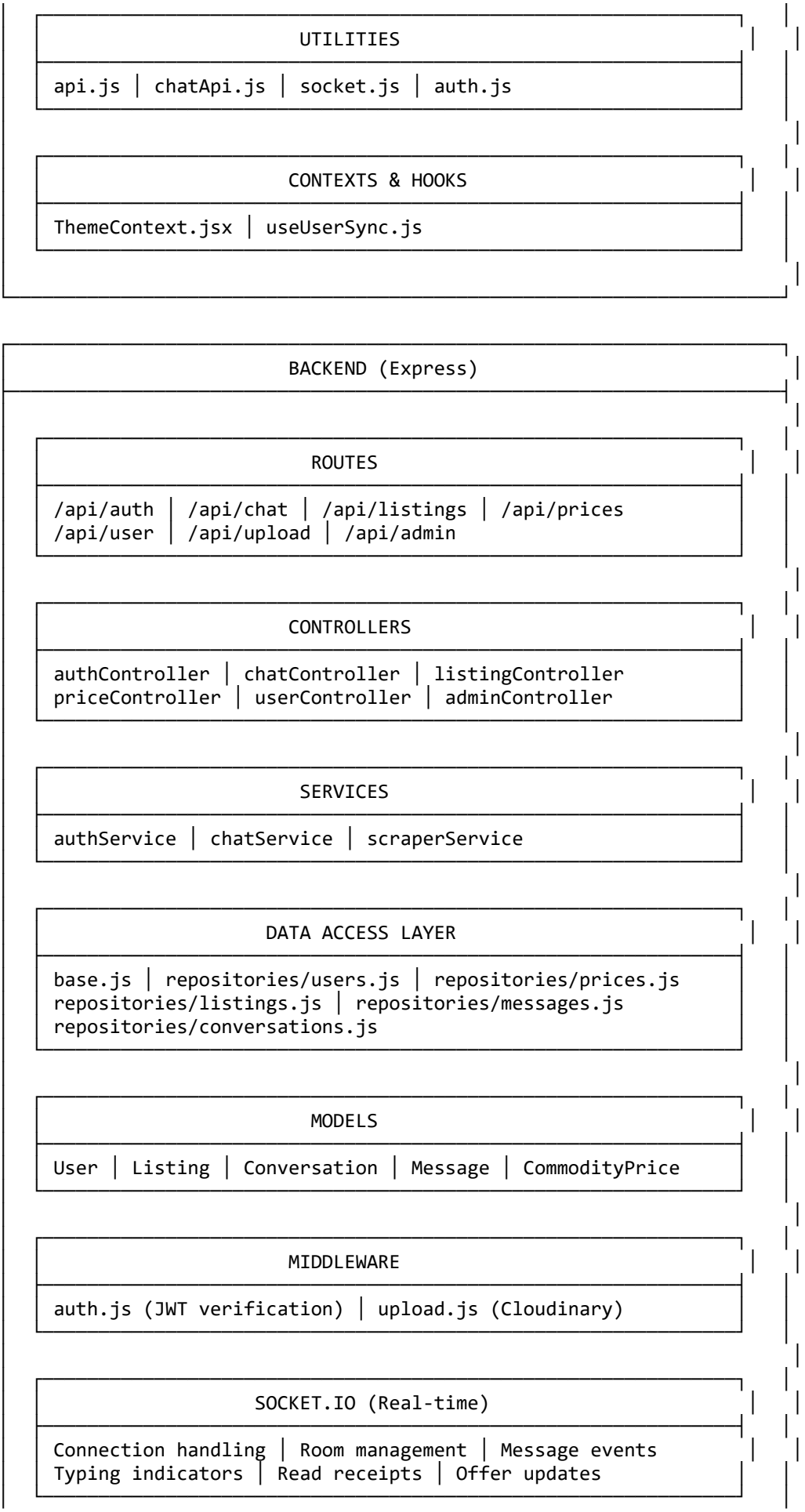
6.1 High-Level Architecture



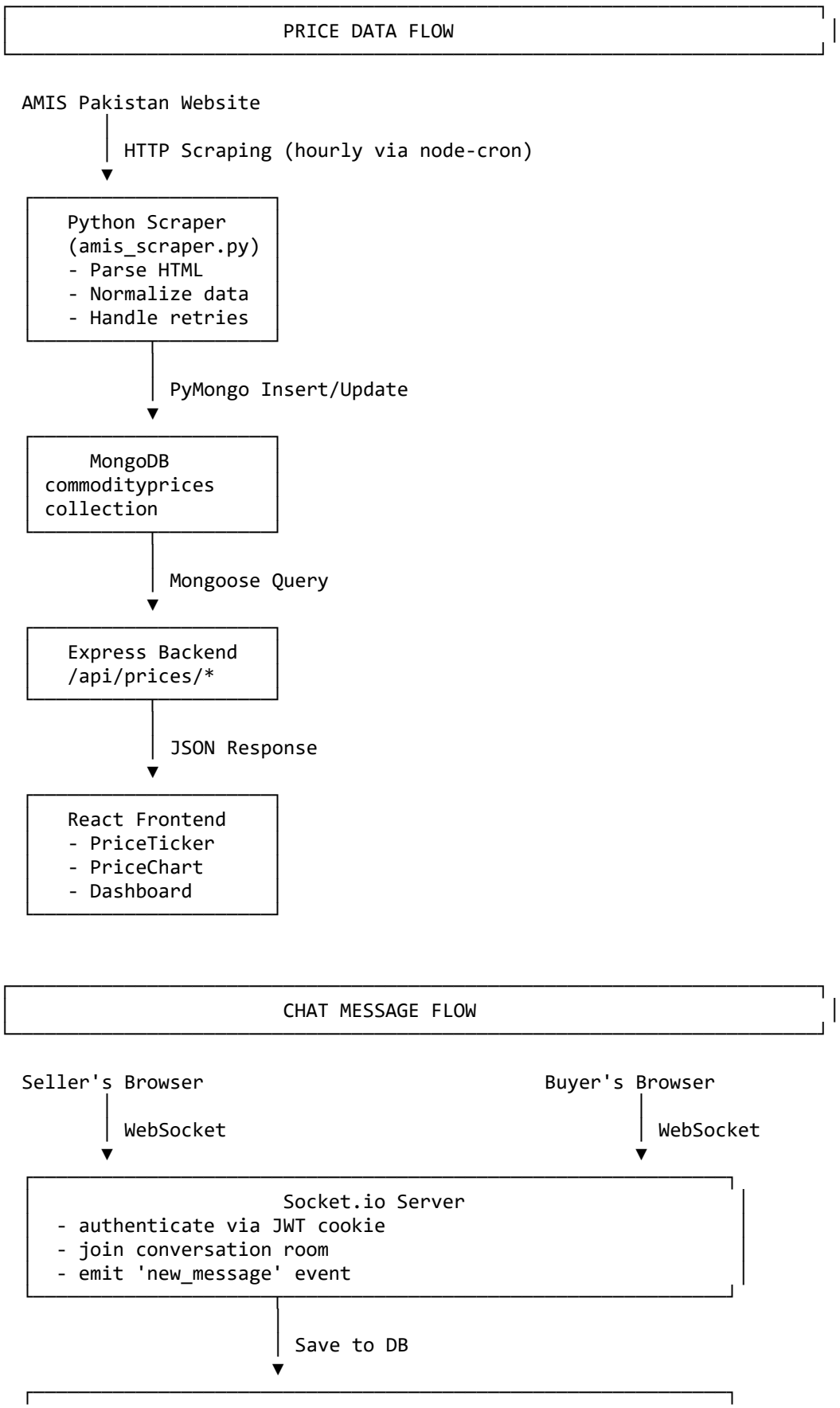


6.2 Component Diagram




UTILITIES
api.js | chatApi.js | socket.js | auth.js
CONTEXTS & HOOKS
ThemeContext.jsx | useUserSync.js
BACKEND (Express)
ROUTES
/api/auth | /api/chat | /api/listings | /api/prices
/api/user | /api/upload | /api/admin
CONTROLLERS
authController | chatController | listingController
priceController | userController | adminController
SERVICES
authService | chatService | scraperService
DATA ACCESS LAYER
base.js | repositories/users.js | repositories/prices.js
repositories/listings.js | repositories/messages.js
repositories/conversations.js
MODELS
User | Listing | Conversation | Message | CommodityPrice
MIDDLEWARE
auth.js (JWT verification) | upload.js (Cloudinary)
SOCKET.IO (Real-time)
Connection handling | Room management | Message events
Typing indicators | Read receipts | Offer updates

6.3 Data Flow Diagram



```
                    MongoDB
                messages collection
```

---

7. Technology Stack

7.1 Frontend Technologies

Technology	Version	Purpose
React	19.x	UI library
Vite	7.x	Build tool
Tailwind CSS	3.x	Utility-first CSS
React Router	7.x	Client-side routing
Recharts	2.x	Price chart visualization
Socket.io Client	4.x	Real-time communication
Lucide React	Latest	Icon library
PropTypes	Latest	Runtime type checking

7.2 Backend Technologies

Technology	Version	Purpose
Node.js	20.x LTS	Runtime environment
Express	5.x	Web framework
Socket.io	4.x	WebSocket server
Mongoose	8.x	MongoDB ODM
JWT	9.x	Token-based auth
bcryptjs	2.x	Password hashing
Cloudinary	2.x	Image storage
node-cron	3.x	Scheduled tasks
cookie-parser	1.x	Cookie handling
cors	2.x	Cross-origin requests

7.3 Database

Technology	Purpose
MongoDB	Primary database
MongoDB Atlas	Cloud hosting (production)

7.4 ML Service (Planned)

Technology	Version	Purpose
Python	3.11+	Runtime
FastAPI	0.100+	API framework
Prophet	1.1+	Time series forecasting
XGBoost	2.0+	Gradient boosting

Technology	Version	Purpose
scikit-learn	1.3+	ML utilities
Pandas	2.0+	Data processing

7.5 Data Scraper

Technology	Purpose
Python	Runtime
BeautifulSoup4	HTML parsing
Requests	HTTP client
PyMongo	MongoDB driver
python-dotenv	Environment variables

7.6 DevOps (Planned)

Technology	Purpose
Docker	Containerization
Docker Compose	Multi-container orchestration
AWS EC2	Cloud hosting
GitHub Actions	CI/CD pipeline
Nginx	Reverse proxy

---

8. Module Descriptions

8.1 User Management Module

Purpose: Handle user registration, authentication, and profile management.

Components: - `authController.js` - Registration, login, logout - `userController.js` - Profile CRUD operations - `authService.js` - JWT token management - `auth.js` (middleware) - Route protection

Features: | Feature | Status | Description | |———|———|————-| | User Registration | ✅ Implemented | Email-based signup with role selection | | User Login | ✅ Implemented | JWT tokens stored in HTTP-only cookies | | Password Hashing | ✅ Implemented | bcrypt with salt rounds | | Profile Management | ✅ Implemented | View/edit profile, avatar upload | | Password Change | ✅ Implemented | Secure password update | | Account Deletion | ✅ Implemented | Soft delete with data cleanup | | Role-based Access | ✅ Implemented | user/admin roles |

API Endpoints:

```
POST   /api/auth/signup          - Register new user
POST   /api/auth/signin          - Login user
POST   /api/auth/logout          - Logout user
GET    /api/auth/me              - Get current user
GET    /api/user/profile         - Get user profile
PUT    /api/user/profile         - Update profile
PUT    /api/user/avatar          - Update avatar
PUT    /api/user/password        - Change password
DELETE /api/user/account         - Delete account
```

8.2 Marketplace Module

Purpose: Enable farmers to list products and buyers to browse/purchase.

Components: - `listingController.js` - CRUD operations for listings - `Listing.js` (model) - Product schema - `ImageUpload.jsx` - Multi-image upload component

Features: | Feature | Status | Description | |———|———|————-| | Create Listing | ✅ Implemented | With multiple images (up to 10) | | Edit Listing | ✅ Implemented | Update all fields | | Delete Listing | ✅ Implemented | Owner-only deletion | | Browse Listings | ✅ Implemented | Paginated results | | Filter by Category | ✅ Implemented | crops, livestock, equipment, etc. | | Search | ✅ Implemented | Title and description search | | My Listings | ✅ Implemented | User's own listings |

Categories Supported: - Crops - Livestock - Equipment - Fertilizers - Seeds - Other

API Endpoints:

```
GET    /api/listings            - Get all listings (paginated)
GET    /api/listings/:id        - Get single listing
POST   /api/listings            - Create listing
PUT    /api/listings/:id        - Update listing
DELETE /api/listings/:id        - Delete listing
GET    /api/listings/my         - Get user's listings
```

8.3 Real-time Chat Module

Purpose: Enable direct communication between buyers and sellers.

Components: - `chatController.js` - Message and conversation management - `chatService.js` - Business logic for chat operations - `socket.js` (config) - Socket.io server setup - `Chat.jsx` - Chat UI page - `MessageBubble.jsx`, `MessageInput.jsx` - Chat components

Features: | Feature | Status | Description | |———|———|————-| | Real-time Messaging | ✅ Implemented | Instant message delivery | | Conversation List | ✅ Implemented | All user conversations | | Message History | ✅ Implemented | Load previous messages | | Typing Indicators | ✅ Implemented | Shows when user is typing | | Read Receipts | ✅ Implemented | Message seen status | | Unread Count | ✅ Implemented | Badge on navbar | | Offer System | ✅ Implemented | Make/accept/reject offers | | Contact Seller | ✅ Implemented | Start chat from listing |

Socket Events:

```
// Client → Server
'join_conversation'     - Join a conversation room
'leave_conversation'    - Leave a conversation room
'send_message'          - Send a new message
'typing'                - User started typing
'stop_typing'           - User stopped typing
'mark_as_read'          - Mark messages as read

// Server → Client
'new_message'           - Receive new message
'user_typing'           - Someone is typing
'user_stopped_typing'   - Someone stopped typing
'offer_updated'         - Offer status changed
'unread_count_updated'  - Unread count changed
```

API Endpoints:

```
GET    /api/chat/conversations              - Get user's conversations
GET    /api/chat/conversations/:id/messages - Get messages in conversation
POST   /api/chat/messages                   - Send message
POST   /api/chat/conversations              - Create/get conversation
PUT    /api/chat/messages/:id/read          - Mark as read
PUT    /api/chat/conversations/:id/offer    - Update offer status
GET    /api/chat/unread-count               - Get unread message count
```

8.4 Price Dashboard Module

Purpose: Display real-time mandi prices and historical trends.

Components: - `priceController.js` - Price data API - `CommodityPrice.js` (model) - Price schema - `PriceChart.jsx` - Interactive chart component - `PriceTicker.jsx` - Scrolling price display

Features: | Feature | Status | Description | |———|———|————-| | Live Price Ticker | ✅ Implemented | Scrolling latest prices | | Price Chart | ✅ Implemented | Recharts visualization | | Commodity Filter | ✅ Implemented | Select commodity type | | City Filter | ✅ Implemented | Select city | | Variety Filter | ✅ Implemented | Select variety | | Date Range | ✅ Implemented | 7/14/30/60/90 days | | Specific Date | ✅ Implemented | Query single date | | Price Change | ✅ Implemented | Show % change |

API Endpoints:

```
GET    /api/prices/commodities              - List all commodities
GET    /api/prices/cities                   - List all cities
GET    /api/prices/varieties/:commodity     - Get varieties for commodity
GET    /api/prices/cities-by-filters        - Get cities for commodity/variety
GET    /api/prices/history                  - Get price history
GET    /api/prices/latest                   - Get latest prices
```

8.5 AMIS Price Scraper Module

Purpose: Automatically fetch and store mandi prices from AMIS Pakistan.

Components: - `amis_scraper.py` - Main scraper script - `scraperService.js` - Node.js process management - `adminController.js` - Admin trigger endpoints

Features: | Feature | Status | Description | |———|———|————-| | Hourly Scraping | ✅ Implemented | via node-cron | | 15 Cities | ✅ Implemented | Punjab coverage | | 6 Commodities | ✅ Implemented | Wheat, Rice, etc. | | Variety Parsing | ✅ Implemented | Extract variety from name | | Price Normalization | ✅ Implemented | Convert to standard units | | Duplicate Handling | ✅ Implemented | Upsert logic | | Retry Logic | ✅ Implemented | Exponential backoff | | Admin Trigger | ✅ Implemented | Manual scrape from admin panel |

Scraped Data Fields:

```
{
  commodity: "Wheat",
  variety: "Desi",
  city: "Lahore",
  price: 4250,
  priceType: "FQP",  // Frequently Quoted Price
  unit: "Rs/40Kg",
  date: "2026-01-19",
  timestamp: "2026-01-19T10:30:00Z"
}
```

8.6 Admin Panel Module

Purpose: Platform management and monitoring.

Components: - `adminController.js` - Admin-only operations - `AdminPanel.jsx` - Admin dashboard UI

Features: | Feature | Status | Description | |———|———|————-| | User Statistics | ✅ Implemented | Total users, registrations | | Listing Statistics | ✅ Implemented | Active listings count | | User Management | ✅ Implemented | View/edit/delete users | | Scraper Control | ✅ Implemented | Trigger manual scrape | | Scraper Status | ✅ Implemented | Check if running |

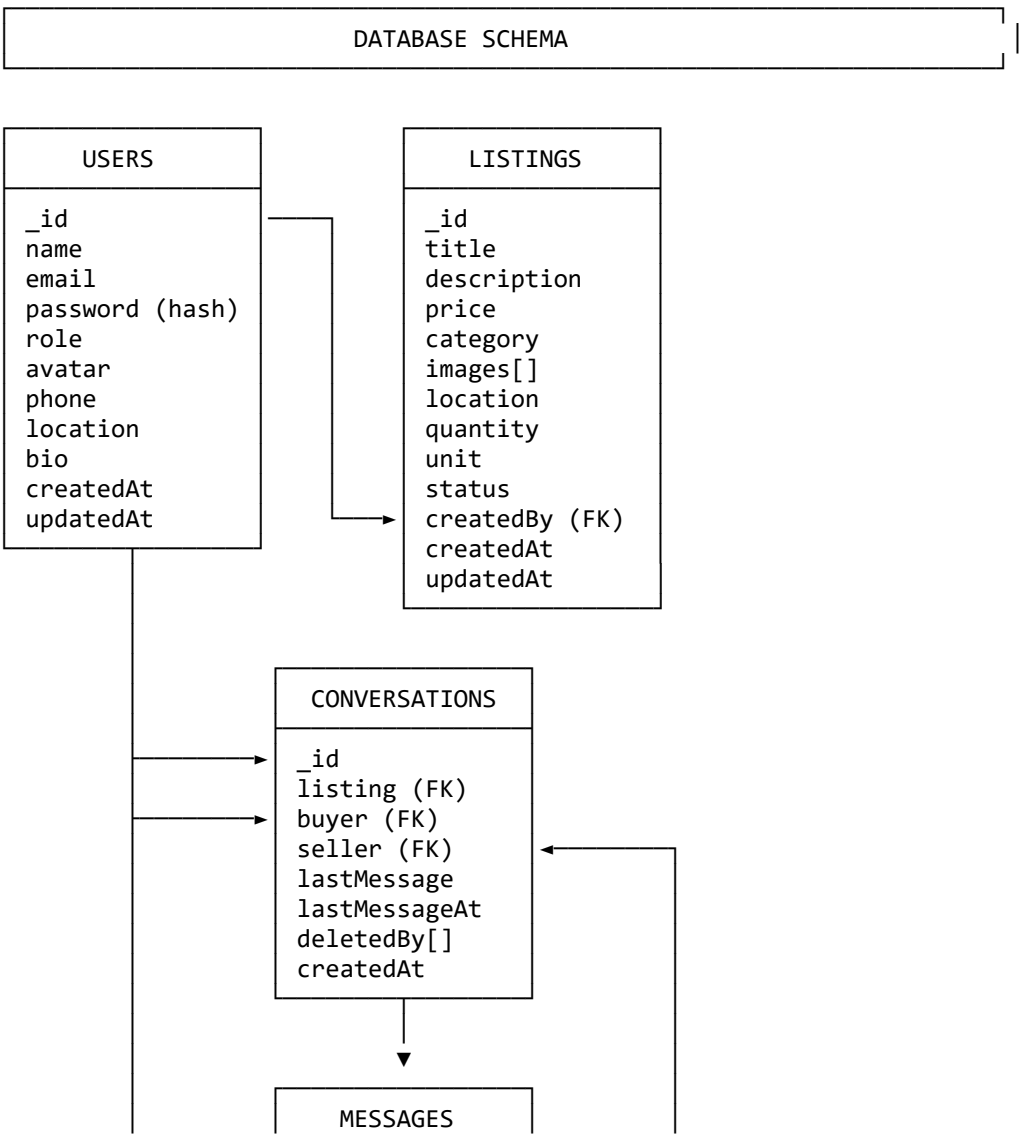
Admin Routes:

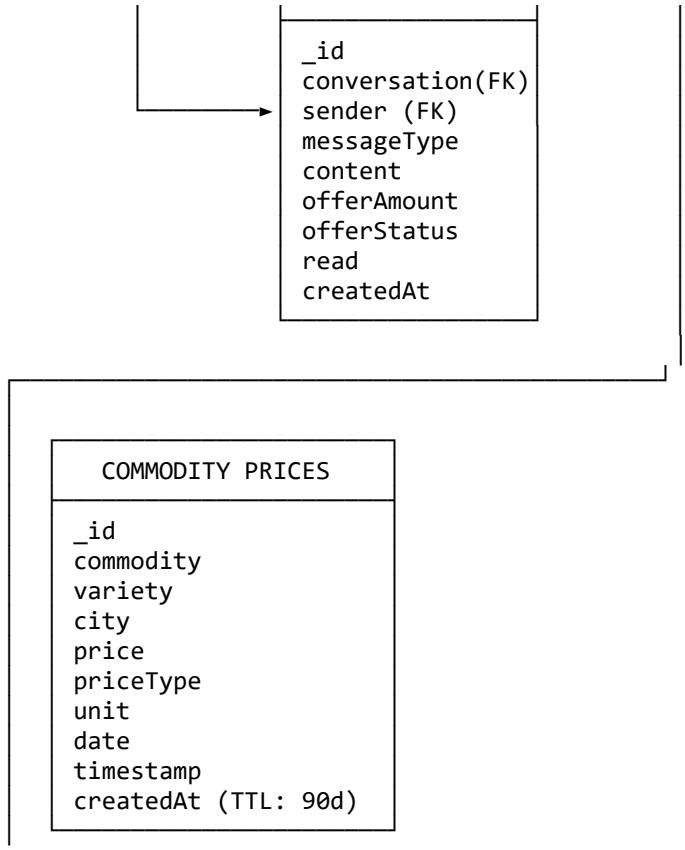
```
GET    /api/auth/stats/users       - Get user statistics
GET    /api/admin/scraper/status   - Check scraper status
POST   /api/admin/scraper/trigger  - Trigger manual scrape
```

---

9. Database Design

9.1 Entity Relationship Diagram





9.2 Collection Schemas

Users Collection

```
const userSchema = new Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 6 },
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
  avatar: { type: String, default: '' },
  phone: { type: String, default: '' },
  location: { type: String, default: '' },
  bio: { type: String, default: '' },
}, { timestamps: true });
```

Listings Collection

```
const listingSchema = new Schema({
  title: { type: String, required: true, trim: true },
  description: { type: String, required: true },
  price: { type: Number, required: true, min: 0 },
  category: {
    type: String,
    required: true,
    enum: ['crops', 'livestock', 'equipment', 'fertilizers', 'seeds', 'other']
  },
  images: [{ type: String }],
  location: { type: String, required: true },
  quantity: { type: Number, required: true, min: 0 },
  unit: { type: String, required: true },
  status: {
```

```
    type: String,
    enum: ['active', 'sold', 'inactive'],
    default: 'active'
  },
  createdBy: { type: Schema.Types.ObjectId, ref: 'User', required: true },
}, { timestamps: true });
```

Conversations Collection

```
const conversationSchema = new Schema({
  listing: { type: Schema.Types.ObjectId, ref: 'Listing' },
  buyer: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  seller: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  lastMessage: { type: String, default: '' },
  lastMessageAt: { type: Date, default: Date.now },
  deletedBy: [{ type: Schema.Types.ObjectId, ref: 'User' }],
}, { timestamps: true });
```

Messages Collection

```
const messageSchema = new Schema({
  conversation: { type: Schema.Types.ObjectId, ref: 'Conversation', required: true },
  sender: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  messageType: {
    type: String,
    enum: ['text', 'offer', 'system'],
    default: 'text'
  },
  content: { type: String, default: '' },
  offerAmount: { type: Number },
  offerStatus: {
    type: String,
    enum: ['pending', 'accepted', 'rejected', null],
    default: null
  },
  read: { type: Boolean, default: false },
}, { timestamps: true });
```

CommodityPrices Collection

```
const commodityPriceSchema = new Schema({
  commodity: { type: String, required: true, index: true },
  variety: { type: String, default: null },
  city: { type: String, required: true, index: true },
  price: { type: Number, required: true },
  priceType: { type: String, default: 'FQP' },
  unit: { type: String, required: true },
  date: { type: Date, required: true, index: true },
  timestamp: { type: Date, default: Date.now },
}, { timestamps: true });

// TTL Index - auto-delete after 90 days
commodityPriceSchema.index({ createdAt: 1 }, { expireAfterSeconds: 90 * 24 * 60 * 60 });
```

9.3 Indexes

```
// Compound indexes for query optimization
db.commodityprices.createIndex({ commodity: 1, city: 1, date: -1 });
db.listings.createIndex({ createdBy: 1, status: 1 });
db.messages.createIndex({ conversation: 1, createdAt: 1 });
db.conversations.createIndex({ buyer: 1, seller: 1 });
```

---

10. API Documentation

10.1 Authentication API

Register User

```
POST /api/auth/signup
Content-Type: application/json

{
  "name": "Ahmad Khan",
  "email": "ahmad@example.com",
  "password": "securepass123",
  "role": "user"
}

Response: 201 Created
Set-Cookie: token=<jwt>; HttpOnly; Secure; SameSite=Lax; Max-Age=604800

{
  "success": true,
  "data": {
    "user": {
      "_id": "...",
      "name": "Ahmad Khan",
      "email": "ahmad@example.com",
      "role": "user"
    }
  }
}
```

Login User

```
POST /api/auth/signin
Content-Type: application/json

{
  "email": "ahmad@example.com",
  "password": "securepass123"
}

Response: 200 OK
Set-Cookie: token=<jwt>; HttpOnly; Secure; SameSite=Lax; Max-Age=604800

{
  "success": true,
  "data": {
    "user": { ... }
  }
}
```

10.2 Listings API

Get All Listings

```
GET /api/listings?page=1&limit=10&category=crops&search=wheat

Response: 200 OK
{
  "success": true,
  "data": {
    "listings": [...],
    "pagination": {
      "total": 50,
      "page": 1,
      "pages": 5,
      "limit": 10
    }
  }
}
```

Create Listing

```
POST /api/listings
Authorization: Cookie (token)
Content-Type: application/json

{
  "title": "Fresh Wheat - 2026 Harvest",
  "description": "High quality wheat from Faisalabad",
  "price": 4500,
  "category": "crops",
  "images": ["https://cloudinary.com/..."],
  "location": "Faisalabad, Punjab",
  "quantity": 500,
  "unit": "kg"
}

Response: 201 Created
{
  "success": true,
  "data": {
    "listing": { ... }
  }
}
```

10.3 Prices API

Get Price History

```
GET /api/prices/history?commodity=Wheat&city=Lahore&variety=Desi&days=30

Response: 200 OK
{
  "success": true,
  "data": [
    {
      "date": "2026-01-19",
      "price": 4250,
      "unit": "Rs/40Kg"
    },
    ...
  ]
}
```

10.4 Chat API

Send Message

```
POST /api/chat/messages
Authorization: Cookie (token)
Content-Type: application/json

{
  "conversationId": "...",
  "content": "Is this still available?",
  "messageType": "text"
}

Response: 201 Created
{
  "success": true,
  "data": {
    "message": { ... }
  }
}
```

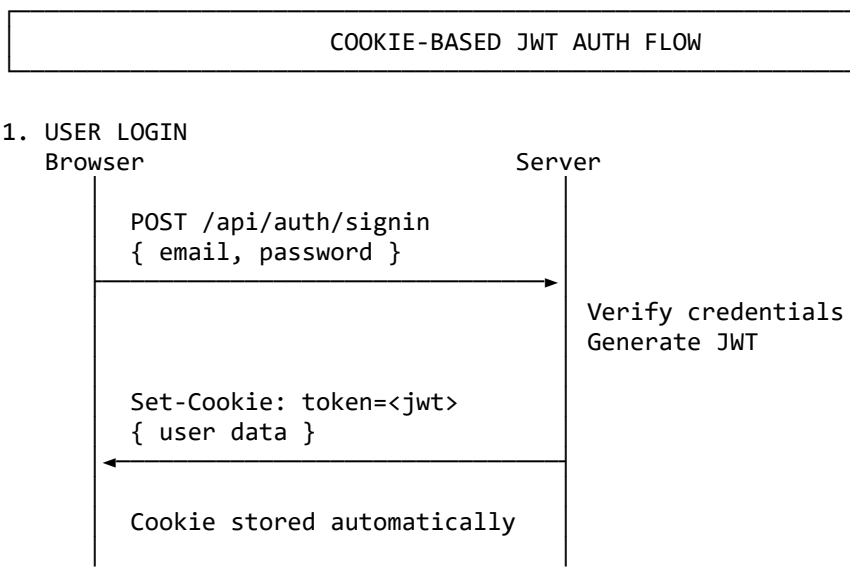
Make Offer

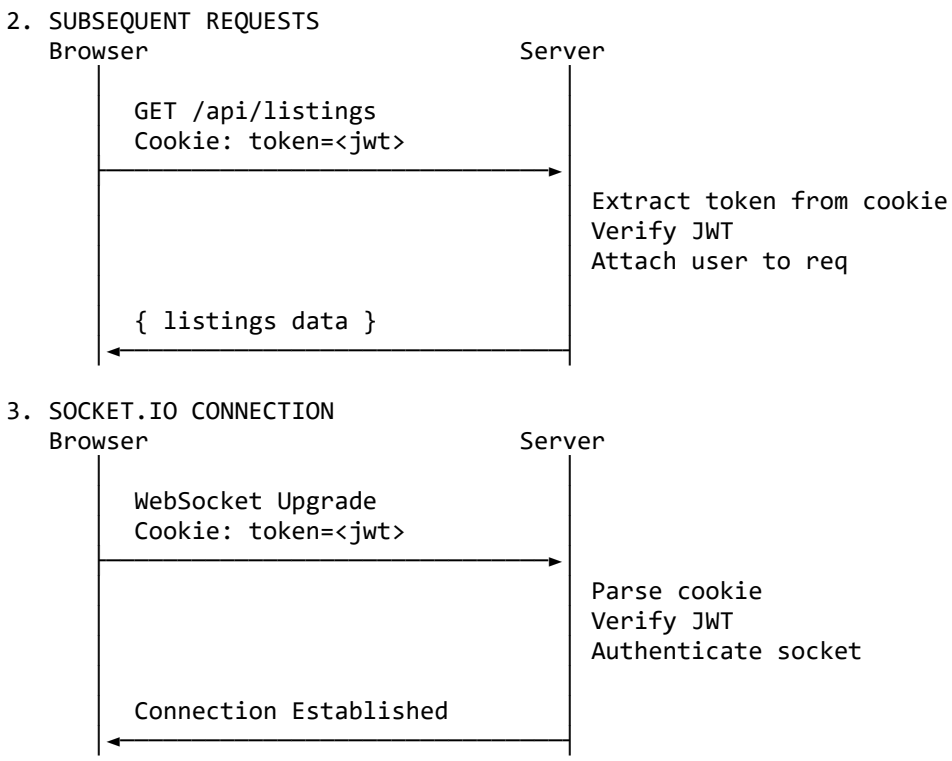
```
POST /api/chat/messages
Authorization: Cookie (token)
Content-Type: application/json

{
  "conversationId": "...",
  "messageType": "offer",
  "offerAmount": 4200
}

Response: 201 Created
```

---

11. Implementation Details

11.1 Authentication Flow





11.2 Real-time Chat Implementation

```
// Server-side Socket.io setup (config/socket.js)
io.use(async (socket, next) => {
  try {
    // Parse cookies from handshake
    const cookies = cookie.parse(socket.handshake.headers.cookie || '');
    const token = cookies.token;

    if (!token) {
      return next(new Error('Authentication required'));
    }

    // Verify JWT
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    socket.userId = decoded.id;
    socket.join(`user_${decoded.id}`);
    next();
  } catch (error) {
    next(new Error('Invalid token'));
  }
});

io.on('connection', (socket) => {
  // Join conversation room
  socket.on('join_conversation', (conversationId) => {
    socket.join(`conversation_${conversationId}`);
  });

  // Handle new message
  socket.on('send_message', async (data) => {
    const message = await chatService.sendMessage(socket.userId, data);
    io.to(`conversation_${data.conversationId}`).emit('new_message', message);
  });
```

```
  // Typing indicators
  socket.on('typing', ({ conversationId }) => {
    socket.to(`conversation_${conversationId}`).emit('user_typing', {
      userId: socket.userId
    });
  });
});
```

11.3 Price Scraper Implementation

```python
# amis_scraper.py - Core scraping logic
class AMISScraper:
    def __init__(self):
        self.base_url = "http://www.amis.pk/"
        self.session = requests.Session()
        self.session.headers.update({
            'User-Agent': 'Mozilla/5.0 (compatible; FarmKonnect/1.0)'
        })

    def scrape_commodity_prices(self):
        """Scrape prices for all target commodities"""
        url = f"{self.base_url}BrowsePrices.aspx?searchType=0"
        response = self.make_request_with_retry(url)
        soup = BeautifulSoup(response.content, 'html.parser')

        prices_data = []
        for commodity_link in self._find_commodity_links(soup):
            commodity_prices = self._fetch_commodity_detail(commodity_link)
            prices_data.extend(commodity_prices)

        return prices_data

    def save_to_mongodb(self, data):
        """Save with upsert logic to prevent duplicates"""
        for item in data:
            filter_doc = {
                "commodity": item["commodity"],
                "variety": item["variety"],
                "city": item["city"],
                "date": item["date"],
                "priceType": item["priceType"]
            }
            update_doc = {"$set": item}
            collection.update_one(filter_doc, update_doc, upsert=True)
```

11.4 Image Upload Flow

```
┌─────────────────────────────────────────────────────────────┐
│                    CLOUDINARY IMAGE UPLOAD                  │
└─────────────────────────────────────────────────────────────┘

1. Client selects images
2. Frontend sends to /api/upload/listing-images
3. Multer middleware processes multipart form
4. Cloudinary SDK uploads to cloud
5. Cloudinary returns secure URLs
6. URLs saved with listing document
```

---

12. ML Price Forecasting System

12.1 Current Status

Status: NOT YET IMPLEMENTED

The ML service scaffold exists (`ml_service/`) but contains no implementation. This section documents the planned implementation approach.

12.2 Data Characteristics

The scraped price data has the following characteristics that influence model selection:

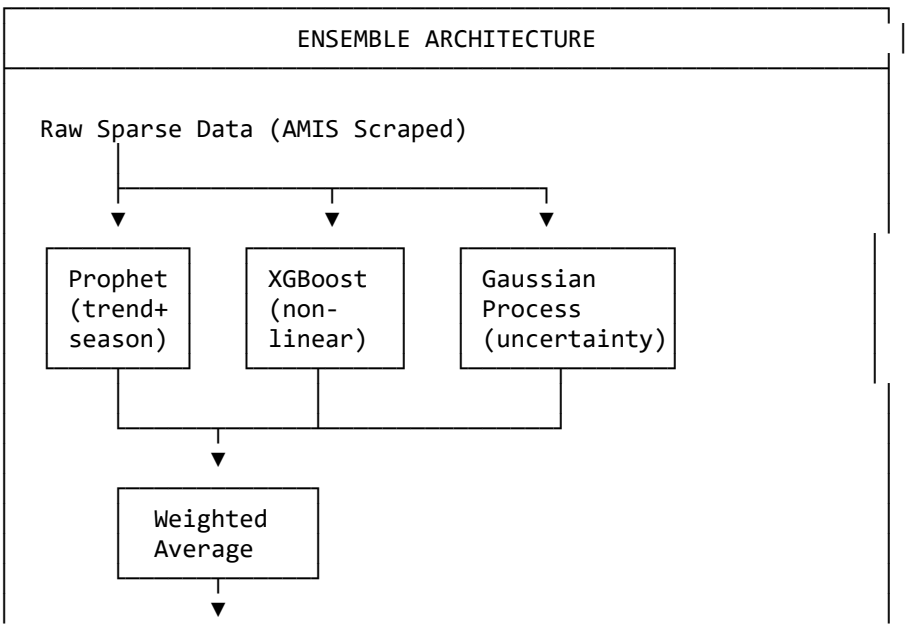
Characteristic	Description	Challenge
Sparse Sampling	Data points not on regular intervals	Cannot use standard ARIMA
Inconsistent Coverage	Not all cities have data for all dates	Missing value handling
Multi-city	15 cities with different data availability	Hierarchical modeling
6 Commodities	Wheat, Rice, Cotton, Sugar, Maize, Flour	Need commodity-specific models
Variety Dimension	Some commodities have varieties	Additional complexity

Example Data Gaps:

```
Wheat prices for Lahore:
- 2025-12-01: 4200
- 2025-12-03: 4180  (2-day gap)
- 2025-12-10: 4250  (7-day gap)
- 2025-12-11: 4255
- 2025-12-25: 4300  (14-day gap)
```

12.3 Recommended Model Architecture

Based on research, we recommend an **Ensemble approach** combining three models:



```
│       7-Day Forecast + Confidence Interval                          │
└──────────────────────────────────────────────────────────────────────┘
```

Model 1: Facebook Prophet

Why: Designed for sparse time series with missing data

```python
from prophet import Prophet

model = Prophet(
    yearly_seasonality=True,      # Capture harvest cycles
    weekly_seasonality=False,     # Prices don't vary by weekday
    changepoint_prior_scale=0.05  # Sensitivity to trend changes
)
model.add_country_holidays(country_name='PK')  # Eid, national holidays
model.add_regressor('fuel_price')               # External factor
model.add_regressor('support_price')            # Government floor
```

Model 2: XGBoost

Why: Captures non-linear relationships with external factors

```python
features = {
    'day_of_year': 'Seasonality',
    'month': 'Monthly patterns',
    'days_since_harvest': 'Post-harvest supply',
    'fuel_price': 'Transportation cost',
    'exchange_rate': 'Import parity',
    'city_encoded': 'Location factor',
    'lag_7_price': 'Recent trend'
}
```

Model 3: Gaussian Process

Why: Provides uncertainty quantification (confidence intervals)

```python
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ExpSineSquared

kernel = (
    RBF(length_scale=30) +                          # Long-term trends
    ExpSineSquared(periodicity=365) +               # Yearly seasonality
    WhiteKernel(noise_level=1)                      # Observation noise
)
```

12.4 External Factors to Include

Factor	Data Source	Impact
Government Support Price	Ministry of Food	Price floor
Fuel/Diesel Prices	OGRA Pakistan	Transportation cost
PKR/USD Exchange Rate	State Bank	Import parity
Rainfall Deviation	PMD/NASA POWER	Yield impact
Ramadan/Eid Dates	Islamic calendar	Demand surge
PASSCO Stock Levels	PASSCO reports	Supply indicator

12.5 Implementation Plan

Phase	Timeline	Deliverable
Phase 1	Week 1-2	Prophet baseline per commodity
Phase 2	Week 3-4	Add external regressors
Phase 3	Week 5-6	XGBoost residual correction
Phase 4	Week 7-8	Gaussian Process uncertainty
Phase 5	Week 9-10	FastAPI endpoints + integration

12.6 Planned API Endpoints

```python
# ml_service/app.py

@app.post("/predict")
async def predict_price(
    commodity: str,      # wheat, rice, cotton, etc.
    city: str,           # Lahore, Multan, etc.
    horizon: int = 7     # Days to forecast
):
    """Return 7-day price forecast with confidence intervals."""
    return {
        "commodity": commodity,
        "city": city,
        "predictions": [
            {"date": "2026-01-20", "price": 4250, "lower": 4100, "upper": 4400},
            {"date": "2026-01-21", "price": 4275, "lower": 4080, "upper": 4470},
            # ... 7 days
        ],
        "model_version": "v1.0.0",
        "last_trained": "2026-01-15"
    }

@app.post("/retrain")
async def retrain_model(commodity: str):
    """Trigger model retraining with latest data."""
    pass

@app.get("/model-metrics")
async def get_metrics(commodity: str):
    """Get model performance metrics."""
    return {
        "mae": 45.2,
        "rmse": 67.8,
        "mape": 1.8,
        "last_evaluated": "2026-01-15"
    }
```

12.7 Evaluation Metrics

Metric	Formula	Target
MAE	Mean Absolute Error	< 50 PKR
RMSE	Root Mean Squared Error	< 75 PKR
MAPE	Mean Absolute Percentage Error	< 5%
Direction Accuracy	% correct up/down predictions	> 70%

13. Security Implementation

13.1 Authentication Security

Measure	Implementation	Status
Password Hashing	bcrypt with 10 salt rounds	✅ Implemented
JWT Tokens	HS256 algorithm, 7-day expiry	✅ Implemented
HTTP-Only Cookies	Prevents XSS token theft	✅ Implemented
Secure Flag	Cookie only sent over HTTPS (prod)	✅ Implemented
SameSite	Lax - prevents CSRF	✅ Implemented

13.2 API Security

Measure	Implementation	Status
CORS	Whitelist allowed origins	✅ Implemented
Input Validation	Mongoose schema validation	✅ Implemented
Rate Limiting	Planned for production	⏳ Planned
Request Sanitization	XSS prevention	⏳ Planned

13.3 Data Security

Measure	Implementation	Status
MongoDB Auth	Username/password authentication	✅ Implemented
TLS Connection	Encrypted DB connections	✅ (Atlas)
Environment Variables	Secrets not in code	✅ Implemented

13.4 JWT Token Structure

```
// Token payload
{
  "id": "user_id",
  "role": "user",        // or "admin"
  "iat": 1737312000,     // Issued at
  "exp": 1737916800      // Expires (7 days)
}

// Cookie settings
{
  httpOnly: true,        // Not accessible via JavaScript
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'lax',       // Prevents CSRF
  maxAge: 7 * 24 * 60 * 60 * 1000  // 7 days
}
```

---

14. Testing Strategy

14.1 Current Status

Testing: NOT YET IMPLEMENTED

No unit tests, integration tests, or E2E tests have been written yet.

14.2 Planned Testing Approach

Unit Testing

- **Framework:** Jest (Node.js), pytest (Python)
- **Scope:** Controllers, services, utilities
- **Coverage Target:** 80%

```
// Example: authService.test.js
describe('AuthService', () => {
  describe('hashPassword', () => {
    it('should hash password correctly', async () => {
      const hash = await authService.hashPassword('password123');
      expect(hash).not.toBe('password123');
      expect(hash.length).toBeGreaterThan(50);
    });
  });

  describe('verifyPassword', () => {
    it('should verify correct password', async () => {
      const hash = await authService.hashPassword('password123');
      const isValid = await authService.verifyPassword('password123', hash);
      expect(isValid).toBe(true);
    });
  });
});
```

Integration Testing

- **Framework:** Supertest
- **Scope:** API endpoints with database
- **Database:** MongoDB in-memory (mongodb-memory-server)

```
// Example: auth.integration.test.js
describe('POST /api/auth/signup', () => {
  it('should register new user', async () => {
    const res = await request(app)
      .post('/api/auth/signup')
      .send({ name: 'Test', email: 'test@test.com', password: 'test123' });

    expect(res.status).toBe(201);
    expect(res.body.data.user.email).toBe('test@test.com');
    expect(res.headers['set-cookie']).toBeDefined();
  });
});
```

E2E Testing

- **Framework:** Cypress or Playwright
- **Scope:** Critical user flows
- **Flows:** Registration, Login, Create Listing, Chat

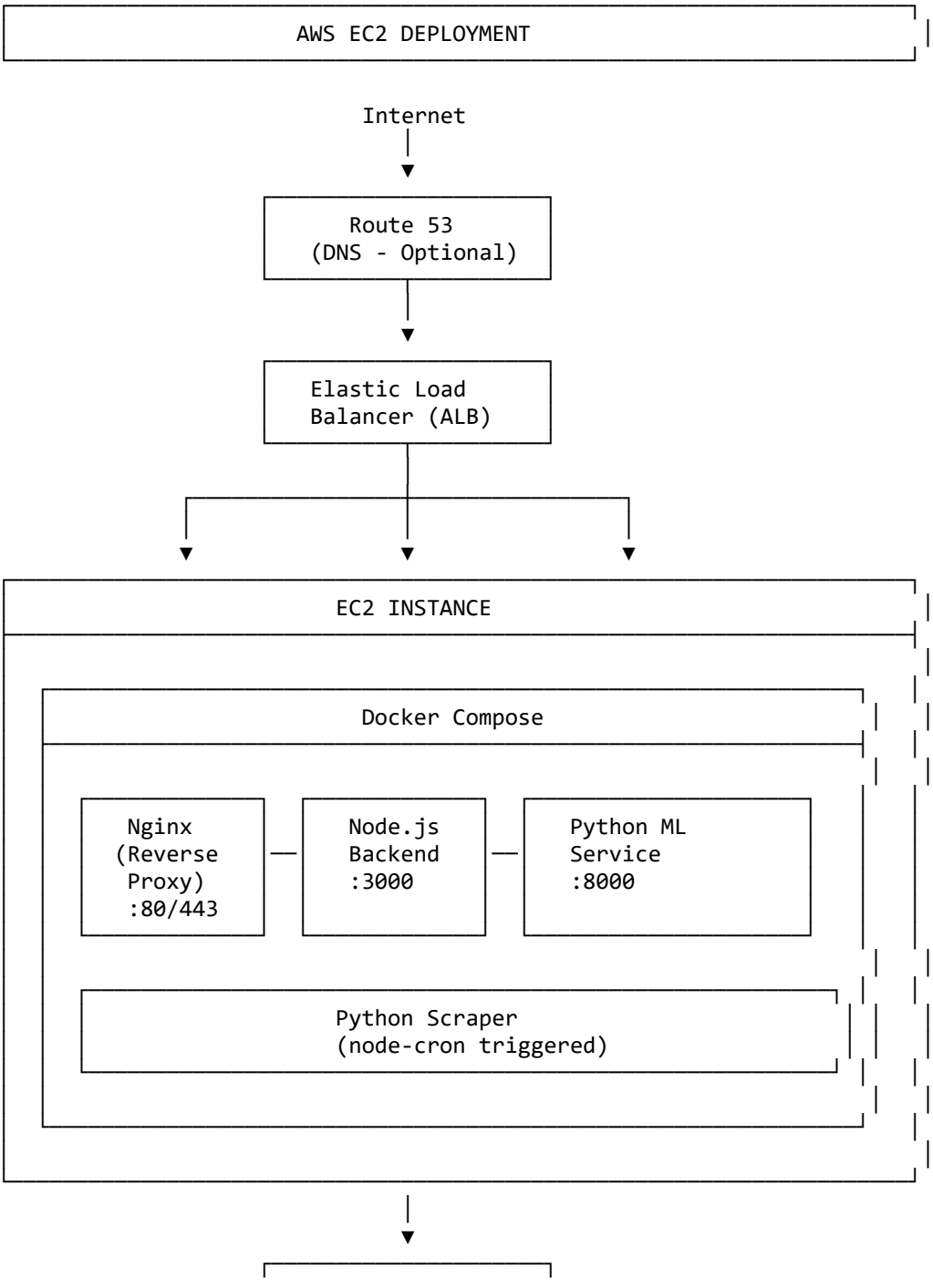
14.3 Testing Timeline

Phase	Tests	Timeline
Unit Tests	Auth, Chat, Listing services	Week 1
Integration Tests	API endpoints	Week 2
E2E Tests	Critical flows	Week 3
ML Model Tests	Forecast accuracy	Week 4

---

15. Deployment Architecture

15.1 Planned Deployment



```
    MongoDB Atlas
    (Managed Cloud)
```

15.2 Docker Configuration

```yaml
# docker-compose.yml
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./frontend/dist:/usr/share/nginx/html
    depends_on:
      - backend

  backend:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      - MONGODB_URI=${MONGODB_URI}
      - JWT_SECRET=${JWT_SECRET}
      - CLOUDINARY_URL=${CLOUDINARY_URL}
    depends_on:
      - ml_service

  ml_service:
    build: ./ml_service
    ports:
      - "8000:8000"
    environment:
      - MONGODB_URI=${MONGODB_URI}

  scraper:
    build: ./scrapper
    environment:
      - MONGODB_URI=${MONGODB_URI}
```

15.3 CI/CD Pipeline

```yaml
# .github/workflows/deploy.yml
name: Deploy to AWS EC2

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run Backend Tests
```

```
        run: cd backend && npm test
      - name: Run Frontend Tests
        run: cd frontend && npm test

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build Frontend
        run: cd frontend && npm run build
      - name: Build Docker Images
        run: docker-compose build

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to EC2
        uses: appleboy/ssh-action@master
        with:
          host: ${{ secrets.EC2_HOST }}
          username: ec2-user
          key: ${{ secrets.EC2_SSH_KEY }}
          script: |
            cd /app
            git pull origin main
            docker-compose down
            docker-compose up -d --build
```

15.4 Environment Variables

```
# Production .env
NODE_ENV=production
PORT=3000

# MongoDB
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/farmkonnect

# JWT
JWT_SECRET=<long-random-string>
JWT_EXPIRES_IN=7d

# Cloudinary
CLOUDINARY_CLOUD_NAME=<cloud-name>
CLOUDINARY_API_KEY=<api-key>
CLOUDINARY_API_SECRET=<api-secret>

# Frontend URL (for CORS)
FRONTEND_URL=https://farmkonnect.pk

# ML Service
ML_SERVICE_URL=http://ml_service:8000
```

---

16. Current Implementation Status

16.1 Summary Table

Module	Status	Completion
User Authentication	✅ Complete	100%
User Profile Management	✅ Complete	100%
Product Listings CRUD	✅ Complete	100%
Marketplace Browse/Filter	✅ Complete	100%
Real-time Chat	✅ Complete	100%
Offer/Negotiation System	✅ Complete	100%
AMIS Price Scraper	✅ Complete	100%
Price Dashboard & Charts	✅ Complete	100%
Admin Panel	✅ Complete	100%
Image Upload (Cloudinary)	✅ Complete	100%
Dark/Light Theme	✅ Complete	100%
ML Price Forecasting	❌ Not Started	0%
Payment Integration	❌ Not Started	0%
Push Notifications	❌ Not Started	0%
Bilingual Support	❌ Not Started	0%
Unit Testing	❌ Not Started	0%
Docker Containerization	❌ Not Started	0%
AWS Deployment	❌ Not Started	0%
CI/CD Pipeline	❌ Not Started	0%

16.2 Overall Progress


```
Core Features:  ████████████████████ 100%
ML Features:                         0%
Payments:                            0%
DevOps:                              0%
Testing:                             0%
```

---

17. Future Work

17.1 Immediate Priorities (Next 4 Weeks)

Priority	Task	Effort
1	Implement ML Price Forecasting	3 weeks
2	Docker Containerization	1 week
3	AWS EC2 Deployment	1 week

17.2 Medium-Term (4-8 Weeks)

Task	Description
Easypaisa/JazzCash Integration	Payment gateway integration for secure transactions

Task	Description
CI/CD Pipeline	GitHub Actions for automated testing and deployment
Push Notifications	Firebase Cloud Messaging for alerts
Unit & Integration Tests	Jest/pytest test suites

17.3 Long-Term Enhancements

Enhancement	Description
Bilingual UI	Urdu/English language switching
Mobile App	React Native or Flutter app
Weather Integration	OpenWeather API for forecasting
Crop Recommendations	AI-based crop selection
Province Expansion	Sindh, KPK, Balochistan coverage
More Commodities	Vegetables, fruits, pulses

17.4 ML Implementation Roadmap

```
Week 1-2:   Prophet baseline model
Week 3-4:   Add external regressors (fuel, exchange rate)
Week 5-6:   XGBoost residual correction
Week 7-8:   Gaussian Process uncertainty
Week 9-10:  FastAPI integration + testing
```

---

18. Conclusion

FarmKonnect represents a significant step toward digitizing Pakistan's agricultural marketplace. The platform successfully addresses key challenges:

Achievements

1. **Real-time Price Transparency**: Live mandi prices from 15 Punjab cities
2. **Direct Trade Platform**: Eliminates middlemen through buyer-seller chat
3. **Modern Tech Stack**: React, Express, MongoDB, Socket.io
4. **Automated Data Collection**: Hourly AMIS scraping
5. **Secure Authentication**: Cookie-based JWT with HTTP-only cookies

Remaining Work

1. **AI Forecasting**: Prophet + XGBoost ensemble for 7-day predictions
2. **Payment Integration**: Easypaisa/JazzCash for secure transactions
3. **Production Deployment**: Docker + AWS EC2 + CI/CD

Impact Potential

- **Farmers**: Access to fair prices, direct buyer connections
- **Buyers**: Verified suppliers, price benchmarking
- **Market**: Increased transparency, reduced information asymmetry

FarmKonnect aims to empower Pakistan's farming community through technology, transparency, and fair trade.

19. References

1. Pakistan Bureau of Statistics. (2024). *Agricultural Statistics of Pakistan*.
2. AMIS Pakistan. http://www.amis.pk/
3. State Bank of Pakistan. (2024). *Agricultural Credit Statistics*.
4. FAO. (2024). *Food Price Index*.
5. React Documentation. https://react.dev/
6. MongoDB Documentation. https://docs.mongodb.com/
7. Socket.io Documentation. https://socket.io/docs/
8. Facebook Prophet. https://facebook.github.io/prophet/
9. XGBoost Documentation. https://xgboost.readthedocs.io/
10. AWS EC2 Documentation. https://docs.aws.amazon.com/ec2/

---

20. Appendices

Appendix A: Environment Setup

```
# Clone repository
git clone https://github.com/your-repo/farmkonnect.git
cd farmkonnect

# Backend setup
cd backend
npm install
cp .env.example .env
# Edit .env with your values
npm run dev

# Frontend setup (new terminal)
cd frontend
npm install
npm run dev

# Scraper setup (new terminal)
cd scrapper
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python amis_scraper.py
```

Appendix B: API Error Codes

Code	Message	Description
400	Bad Request	Invalid input data
401	Unauthorized	Missing or invalid token
403	Forbidden	Insufficient permissions
404	Not Found	Resource doesn’t exist
409	Conflict	Duplicate resource
500	Internal Server Error	Server-side error

Appendix C: Database Indexes

```
// Recommended indexes for production
db.users.createIndex({ email: 1 }, { unique: true });
db.listings.createIndex({ createdBy: 1, status: 1 });
db.listings.createIndex({ category: 1, createdAt: -1 });
db.commodityprices.createIndex({ commodity: 1, city: 1, date: -1 });
db.messages.createIndex({ conversation: 1, createdAt: 1 });
db.conversations.createIndex({ buyer: 1, seller: 1 });
```

Appendix D: Glossary

Term	Definition
Mandi	Agricultural wholesale market
آڑھتی (Aarthi)	Commission agent/middleman
AMIS	Agricultural Marketing Information System
PASSCO	Pakistan Agricultural Storage & Services Corporation
FQP	Frequently Quoted Price
Rabi	Winter cropping season (Oct-Apr)
Kharif	Summer cropping season (Apr-Oct)
Maund	Traditional weight unit (40 kg)

Document Version: 1.0 Last Updated: January 19, 2026 FarmKonnect - AI-Powered Agricultural Marketplace